Diffractive Optical Computing: Simulating Light-Based Neural Networks via Free-Space Wave Propagation and Training a single learnable diffractive layer

Objective: Phase 1-Build and validate a numerical free-space light propagation simulator based on the angular spectrum method, as the foundation for a diffractive neural network (D²NN) — an optical system that performs computation (e.g., image classification) via light diffraction through trained phase masks rather than conventional electronic computation. This deliverable is a validated propagation engine.

Phase 2: Implement a single learnable phase mask, optimized via automatic differentiation (PyTorch), to perform a simple routing task — a bright input spot at one of two positions should be routed by the mask to one of two corresponding output regions. This is the smallest possible proof-of-concept that a diffractive layer can be trained like a neural network layer, and is a direct stepping stone toward a full diffractive optical neural network (D²NN).

Tools/technologies:

- Python, NumPy/SciPy (FFT-based angular spectrum propagation)
- Matplotlib (visualizing intensity/phase patterns)
- PyTorch (autodiff to train phase mask)

Background:

Diffraction and free-space light propagation are commonly analyzed using the Fraunhofer and Fresnel approximations, which are valid only in specific regimes — far-field and paraxial (small-angle), respectively. The angular spectrum method (ASM) offers a more general approach: it solves the propagation problem exactly, without assuming a far-field distance or restricting the wave to small angles, by decomposing the input field into its constituent plane-wave components (via a 2D Fourier transform), propagating each component according to its exact phase evolution (derived from the Helmholtz equation), and reconstructing the propagated field via an inverse Fourier transform. This makes ASM applicable across near-field and far-field regimes alike, at the cost of being computationally heavier than the closed-form Fraunhofer/Fresnel formulas.

A diffractive optical neural network (D²NN) performs computation using a series of phase masks and the free-space propagation between them. The key idea behind training such a system is that the propagation step is built entirely from differentiable operations (Fourier transforms and element-wise multiplication by a known transfer function). Because of this, the gradient of an error measure with respect to every pixel of a phase mask can be computed by automatic differentiation, and the mask can be optimized with the same gradient-based methods used to train conventional neural networks (Lin et al., *Science*, 2018). The angular spectrum engine validated in Phase 1 provides exactly this physical forward model.

This phase takes the smallest possible step in that direction: a single trainable phase mask, trained for a binary routing task in which light entering from one of two input positions must be directed to one of two output regions. A single layer with two classes is deliberately minimal. It tests whether the full training pipeline works (a differentiable propagation model, a loss function, and a gradient-based optimizer) before the added complexity of multiple layers and larger datasets. The propagation engine was reimplemented in PyTorch to make this possible, since PyTorch can compute gradients through complex-valued FFT operations automatically.

This project's long-term aim is to build toward a diffractive optical neural network (D²NN) — a system that performs computation (e.g. image classification) using trained phase masks and free-space diffraction rather than conventional electronic hardware. Since any such system depends fundamentally on accurately simulating how light propagates between layers, a validated ASM-based propagation engine is the necessary first building block. This report covers that first phase 1 and 2: implementing and validating the propagation engine itself, and training a single learnable layer.

Derivation:

Deriving the wave equation from the maxwell's equation:

Faraday's Law: ∇ x E = - ∂B/ ∂t

Ampere- Maxwell law: ∇ x B = $\mu o J + \mu o \varepsilon o \, \partial E / \partial t$

In vacuum with no charge —-> no current

Hence, J = 0, $\therefore \nabla$ x B = $\mu o \, \varepsilon o \, \partial E / \partial t$

$\therefore \nabla$ x (∇XE) = - ∂ (∇XB) / ∂t

On solving this we get: $\nabla^2 E = \mu o \, \varepsilon o \, \partial^2 E / \partial t^2$ —-------> Maxwell's wave equation

Helmholtz – time independence:

E (x,y,z,t) = U(x,y,z) exp(-iwt)

$\therefore$ ∇^2 U + k^2 U = 0 —-----> where k= wave number = w/v

This is the time independent helmholtz equation.

Substituting a plane wave solution, U = exp(i(k_xx+k_yy+k_zz)), into the Helmholtz equation gives the constraint:

$k_x^2 + k_y^2 + k_z^2 = k^2$

Solving for k_z:

$k_z = \text{sqrt}\ (k^2 - k_x^2 - k_y^2)$

Converting to spatial frequency ($f_x = k_x/2\pi$, $f_y = k_y/2\pi$) gives the angular spectrum transfer function:

$H(f_x,f_y) = \exp[i(2\pi/\lambda)z*\text{sqrt}(1 - (\lambda f_x)^2 - (\lambda f_y)^2)]$

When $(\lambda f_x)^2 + (\lambda f_y)^2 < 1$, k_z is real and the component propagates; when this exceeds 1, k_z becomes imaginary and the component is evanescent, decaying exponentially with z rather than propagating.

Angular Spectrum Method: the field U(x,y,0) is Fourier-transformed into its plane-wave components, each component is multiplied by H(f_x,f_y) to apply the correct propagation phase, and an inverse Fourier transform reconstructs the propagated field U(x,y,z).

Method Phase 1:

The optical field is represented as a 2D complex array, constructed on a spatial grid (NumPy `meshgrid`) with physical extent and pixel spacing chosen to satisfy two constraints: fine enough spacing to resolve the wavelength, and a large enough total window to accommodate the field's spread during propagation without wraparound artifacts. Amplitude and phase are combined into a single complex field via Euler's formula: field = amplitude × exp(i·phase).

The propagation engine (`propagate()`) implements the angular spectrum method as follows: the input field is Fourier-transformed (`np.fft.fft2`) into its plane-wave components; the correct frequency values for this transform are obtained via `np.fft.fftfreq`, which accounts for the FFT's native (corner-based) frequency ordering, each component is multiplied by the transfer function H(f_x,f_y,z) derived above; evanescent components (where the argument of the square root in k_z is negative) are handled using `np.emath.sqrt`, which correctly returns imaginary results rather than erroring, the result is inverse-Fourier-transformed (`np.fft.ifft2`) to obtain the propagated field.

Phase 2:

The Phase 1 propagation function was reimplemented in PyTorch so that gradients can flow through it. The calculation is unchanged, but double precision (float64/complex128) is required. The transfer function involves subtracting small spatial-frequency terms from a very large wavenumber term ($k^2 \sim 10^{14}$), and single precision gave results that differed from the

NumPy version by many orders of magnitude. With double precision, the PyTorch version reproduced the Phase 1 Airy and single-slit comparisons.

Task: Two input fields were defined, each a Gaussian spot, one on the left and one on the right of the grid center. The output plane was divided into a top half (region 1) and a bottom half (region 2). The goal is for input A to be routed to region 1 and input B to region 2. The regions were split top/bottom, not left/right, so that the untrained system starts at chance level. Both spots sit on the boundary between the two regions, so an untrained mask splits each input's light about 50/50. A left/right split would have the task partly solved before training starts.

Trainable mask: A single phase mask, the same size as the grid, was defined as a trainable parameter, initialized to zero. Each input is multiplied by exp(i·mask), propagated to the output plane with the Phase 1 engine, and its intensity is computed.

Loss function: For each input, the loss is the fraction of its output intensity landing in the wrong region. The total loss is the sum over both inputs, so it ranges from 0 (perfect routing) to 1 (all light misrouted).

Training: Gradients of the loss with respect to every mask pixel were computed by automatic differentiation, and the mask was updated with the Adam optimizer (learning rate 0.01) for 1000 iterations.

Pseudo code 1:

```
FUNCTION propagate(field, wavelength, z, dx, dy):
    N = size of field (assumes square grid)

    fx, fy = spatial frequency values for grid of size N, spacing dx, dy
    FX, FY = 2D frequency grid from fx, fy

    k = 2*pi / wavelength
    kz = sqrt(k^2 - (2*pi*FX)^2 - (2*pi*FY)^2)
        IF the value under the square root is negative:
            kz becomes imaginary (evanescent component, decays with z)

    H = exp(i * kz * z)      # propagation phase for each frequency component

    field_freq = FFT(field)              # decompose into plane waves
    propagated_freq = field_freq * H     # apply propagation phase to each
    propagated_field = Inverse_FFT(propagated_freq)   # reconstruct field

    RETURN propagated_field
```

Pseudo code 2:

```
Define two input fields: a light spot on the left (A) and on the right (B)
Define two output regions: top half (region 1) and bottom half (region 2)
```

Initialize a phase mask (all zeros) as a trainable parameter

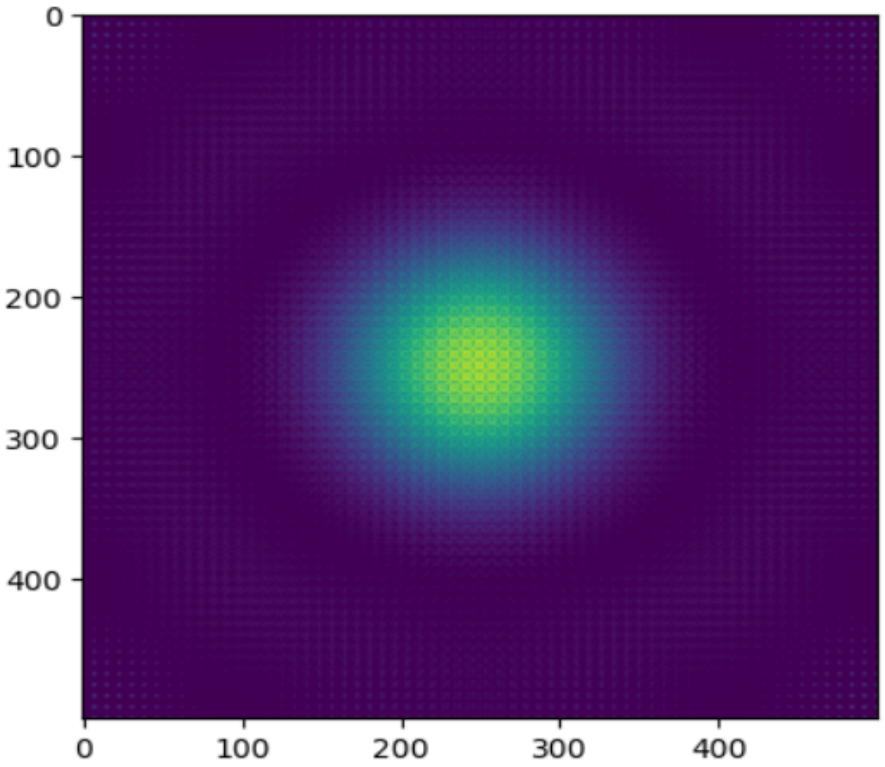
FUNCTION loss(mask):
  For input A:
    Apply the mask to the field
    Propagate the field to the output plane
    Measure the fraction of light landing in region 2 (wrong region)
  For input B:
    Apply the mask to the field
    Propagate the field to the output plane
    Measure the fraction of light landing in region 1 (wrong region)
  RETURN the sum of both wrong-region fractions

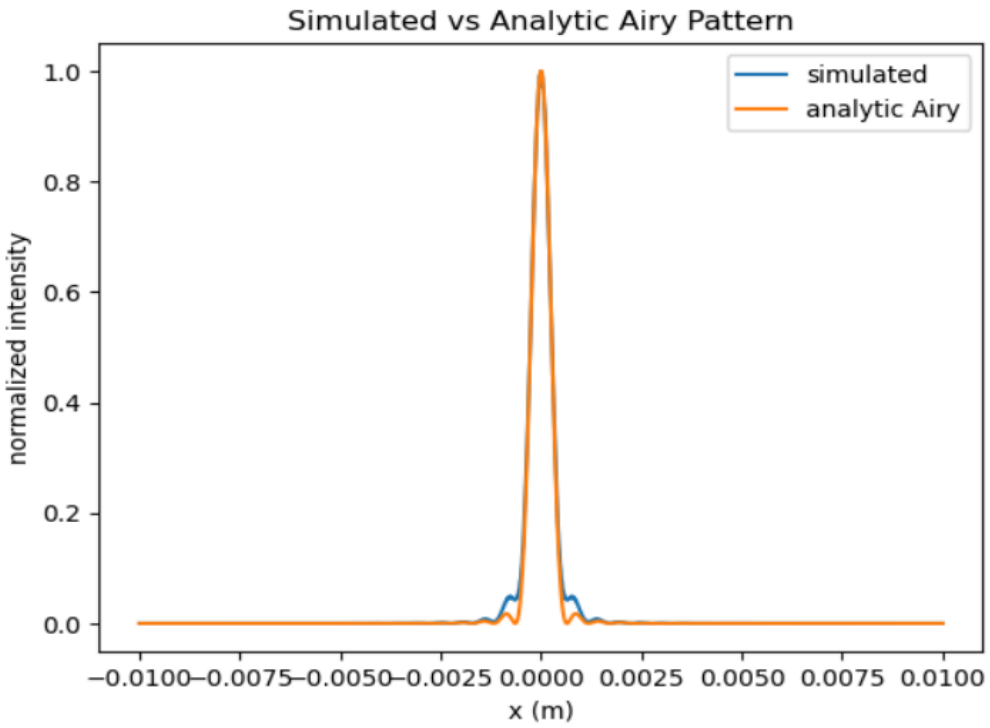
REPEAT for a fixed number of iterations:
  Compute the loss for the current mask
  Compute how the loss changes with respect to each mask value
  Adjust the mask slightly in the direction that reduces the loss

Compare the light distribution for both inputs before and after training

Results — Case 1: Circular Aperture (Airy Pattern)

1. Building a circular aperture: R = sqrt(X**2 + Y**2)
2. Radius : r= 0.0006, relative to grid space
3. Propagated to a far field to see an Airy pattern.
   Z > radius^2 /λ
   Z > 0.0006/633e-9 = 0.57
   Therefore propagation field must be greater than 0.57
4. Comparing the simulated airy pattern with the analytical pattern.
   I (θ) = Io [2J1(u)/ u]^2 —--> u= ka sinθ, k= 2pi/λ, J1= bessel function of order 1
   At θ = 0, I(θ)= Io = max
   Why compare: CIrcular aperture ≠ exactly airy pattern.
5. By importing scipy.special in python we use the bessel function.
6. Calculated intensity using the formula, then normalized both the simulated and the analytical formula
7. Plotted the graph of analytical vs simulated airy pattern.
8. The simulated intensity pattern closely matched the analytic Airy formula in both central lobe width and ring spacing. A small high-frequency ripple was observed on the simulated curve, attributed to angular-spectrum aliasing at large propagation distances — a known numerical artifact rather than a physical discrepancy.
9. Using a smaller propagation distance (closer to the far-field threshold) reduced the ripple, and this comparison uses that adjusted distance.



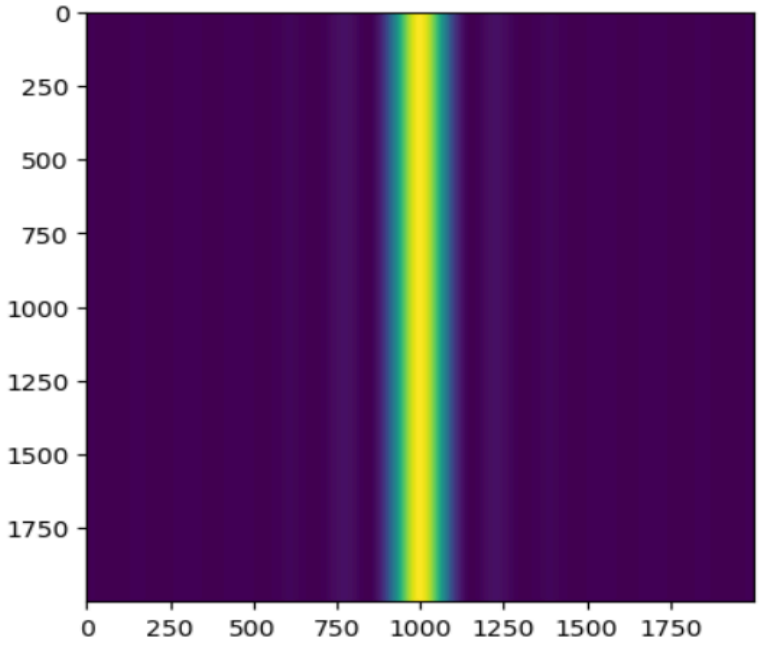


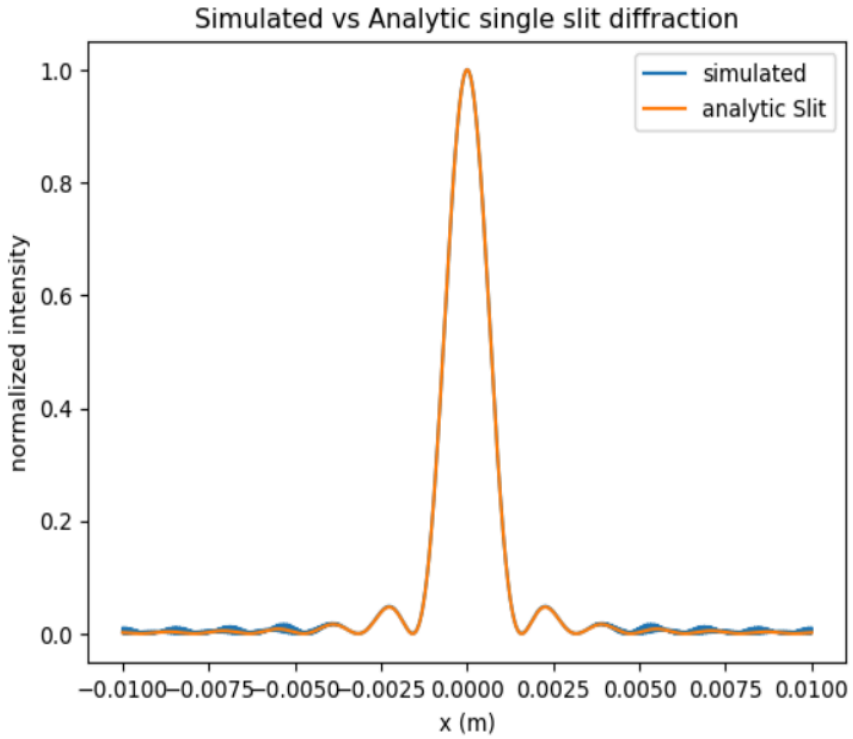
Case 2: Single Slit (Fraunhofer Pattern)

1. Diffraction through a slit (simulated):
   Based on the grid size, chosen halfwidth= 0.0001
   Propagated intensity and simulate single slit diffraction.
2. Analytical calculation:

Z > fullwidth^2 / λ
Z > 0.0632

3. Calculated the analytical single slit using
   I ∝ (sin β/β)^2, —-> β = pi*d*sinθ





Case 3: Reversibility Check

The idea: to propagate a field at distance z then propagate it back to (-z), should get the same result as that of the original field, – if nothing is lost or absorbed – propagation is reversible.

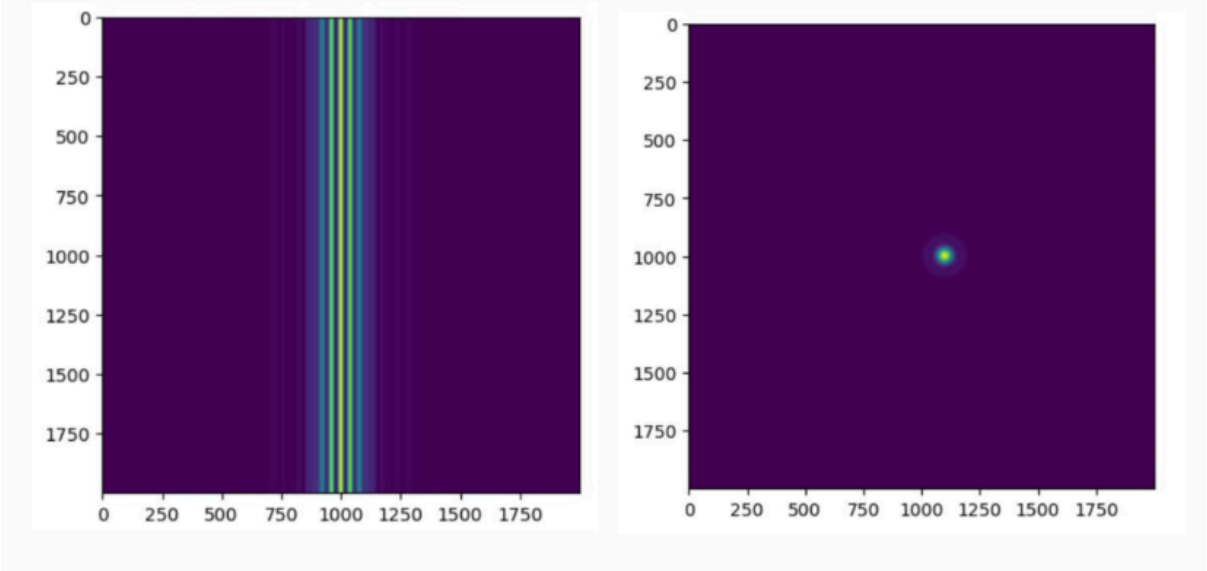
Tests internal consistency – genuine bug (sign error, wrong conjugate) could pass the airy/slits tests but fail this one, hence the independent check is important, not a repetition of what's already done.

1. Made a propagation through z and then back to (-z)
2. Comparing this to the original field.

Got a small difference of `5.560073843692884e-16 in the propagation`
And the difference of `1.9349036575769666e-34  in the mean squared value.`

As additional confidence checks:

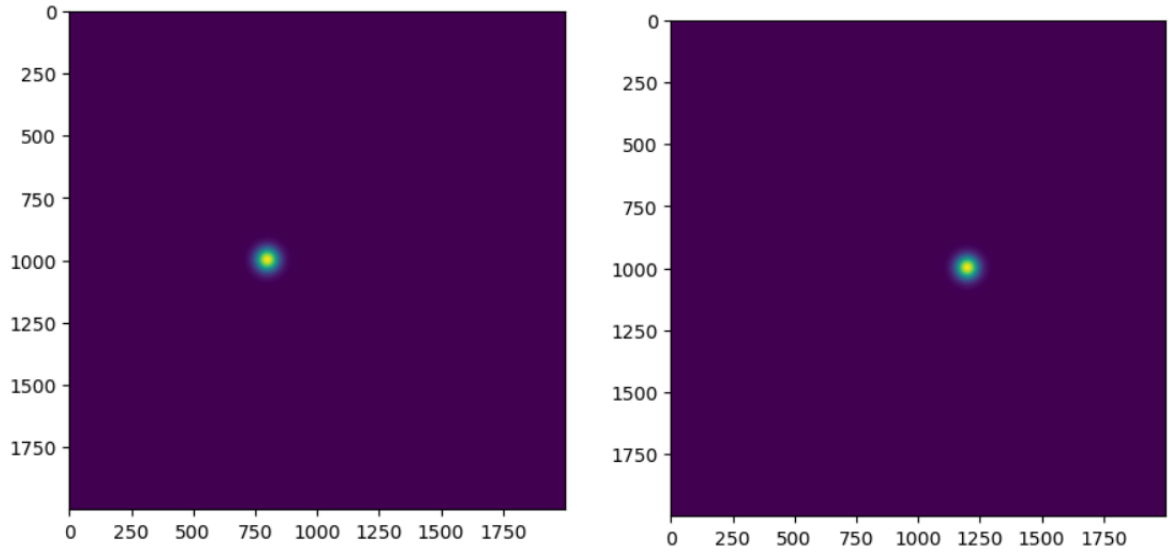
Two further test cases were simulated without formal analytic comparison. A double-slit aperture produced the expected interference pattern — closely spaced fringes within an overall diffraction envelope — consistent with two-slit interference theory. An off-axis circular aperture was also tested to verify the propagation engine correctly handles non-centered input fields; the diffraction pattern shifted appropriately with the aperture's position. At large offsets, the simulation exhibited wraparound artifacts (light reappearing at the opposite edge of the grid), consistent with the periodic boundary assumption inherent to FFT-based methods — a useful, reproducible demonstration of this known limitation of the angular spectrum method on a finite grid.



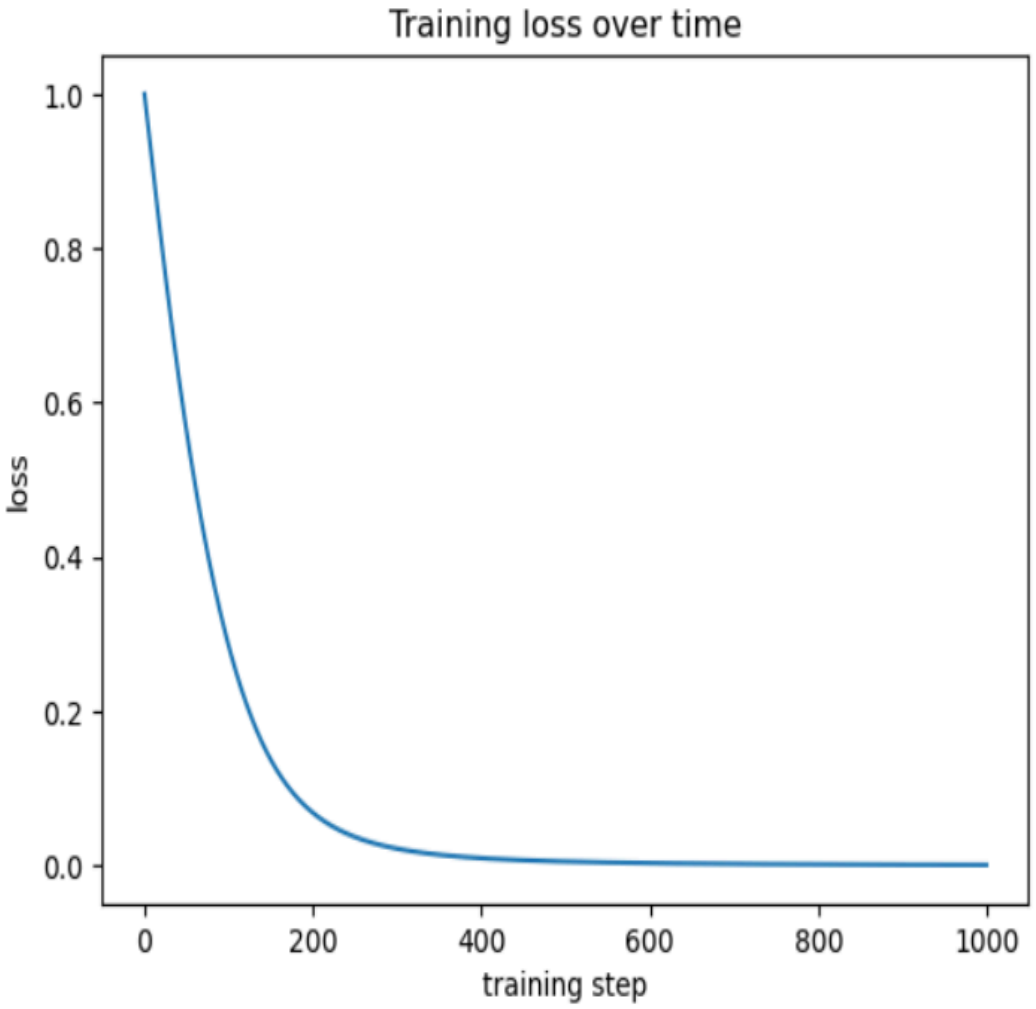
The trained mask:

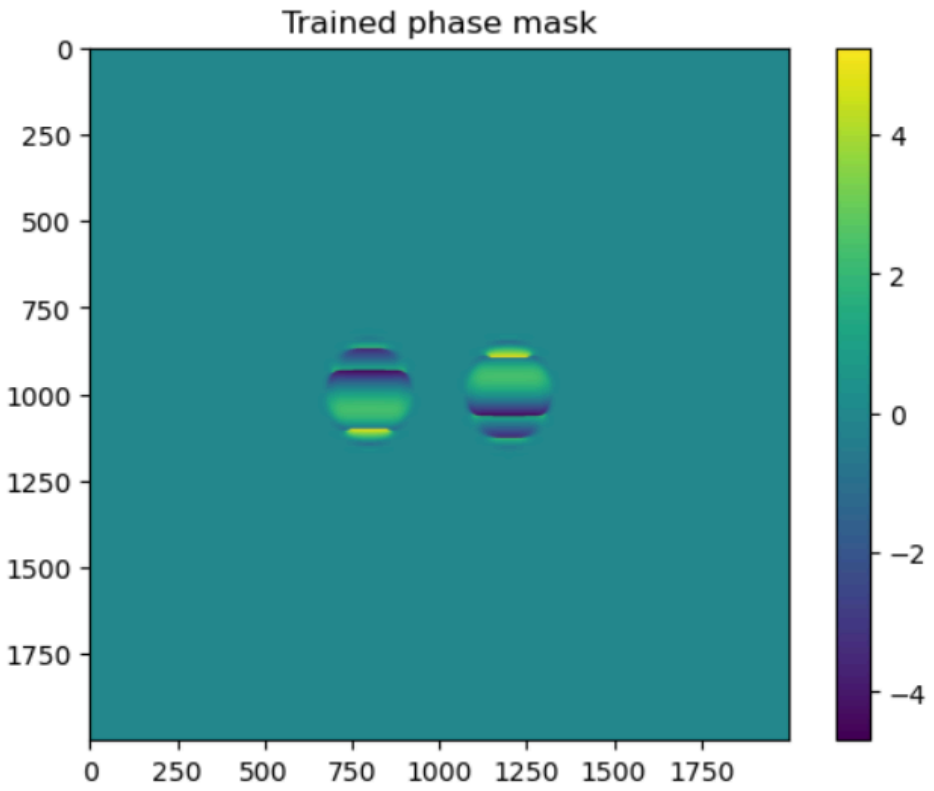
The figure below shows the two input fields (Gaussian spots on the left and right of the grid center) and the two output regions (top and bottom halves).

Before training: With the mask initialized to zero, the mask has no effect (exp(i·0) = 1), and each input's light splits evenly between the two regions: about 1961.5 in each region for both inputs. The initial loss is 1.0, which is chance-level routing (a perfect system would give 0, and complete misrouting would give 2).



Training: The loss decreased steadily over training, from 1.0 to about `0.280427` after 100 iterations and to `0.0009566` after 1000 iterations (Figure of loss curve). The trained mask has no simple geometric structure, which is typical of trained diffractive masks.

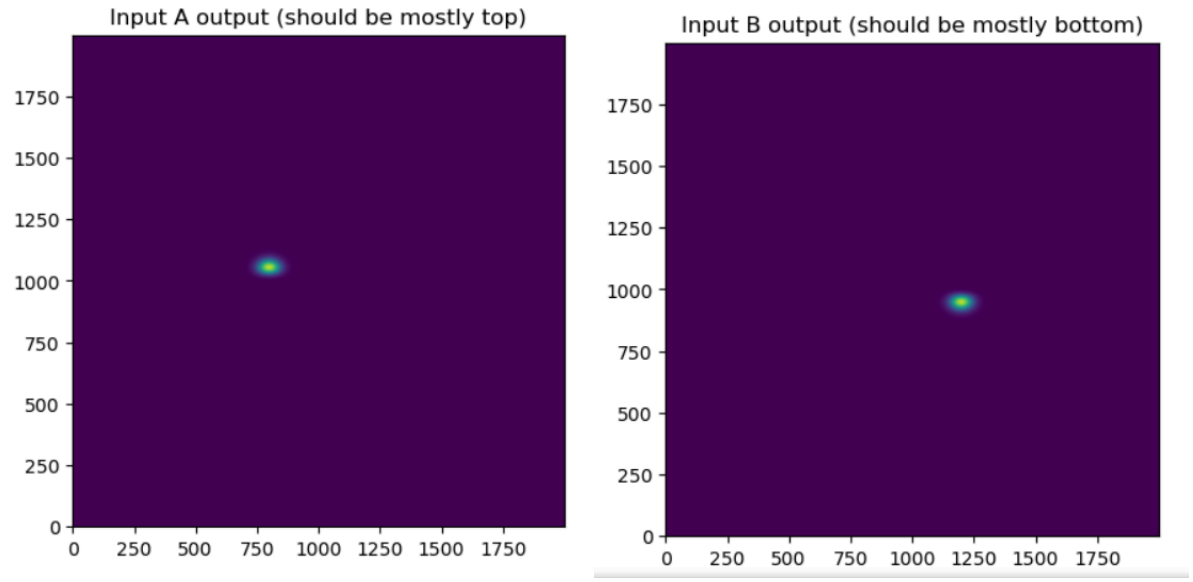




After training: The light is now almost entirely routed to the correct region for both inputs:

- Final loss after 1000 steps: about 0.00096 (starting from 1.0 before training).
- Fraction of light in the correct region: 3921.19 out of 3923.07, which is 99.95%.

	Input A → top	**Input A → bottom**	**Input B → top**	**Input B → bottom**
Before	`1961.5324039589793`	`1961.5324039589793`	`1961.5324039589793`	`1961.5324039589793`
After	`3921.188305237279`	`1.8765026806801364`	`1.876502680680137`	`3921.1883052372787`



Over 99.95% of each input's light ends up in its target region, compared with 50% before training. This shows that a single trained phase mask can perform binary routing based on input position, using only free-space diffraction.

Limitations:

The angular spectrum method, as implemented here, is subject to a few known numerical limitations arising from its use of a finite, discretely sampled grid.

- First, grid design involves a tradeoff between spatial resolution and field of view: the grid spacing must be fine enough to resolve the optical wavelength, while the total physical window must be large enough to contain the field as it spreads during propagation. Insufficient window size leads to wraparound aliasing, where light exiting one edge of the grid reappears at the opposite edge — an artifact of the periodic boundary assumption implicit in the FFT, not a physical effect. This was directly observed when testing an off-axis aperture at large offsets.
- Second, at large propagation distances, the transfer function H(fx,fy) oscillates rapidly as a function of spatial frequency; when this variation is not adequately sampled by the grid, a high-frequency ripple artifact appears in the propagated intensity pattern. This was observed in the circular aperture validation and was mitigated by using a propagation distance closer to the far-field threshold rather than far beyond it.

  Both limitations are well-documented characteristics of FFT-based angular spectrum implementations, and were managed in this work through careful selection of grid size and propagation distance rather than requiring changes to the method itself.

Also the trained mask results above come with several limitations, which also point to directions for further work.

- The mask was trained and tested on the same two inputs, so this shows the optimization works but not that it generalizes.
- The phase values are unconstrained, with no limit on range or resolution, so this isn't yet a fabricable mask.
- Each training step runs two propagations on a 2000×2000 grid, so training is slow on a CPU.

Conclusion:

This phase of the project successfully implemented and validated a free-space light propagation simulator based on the angular spectrum method. The propagation engine was derived from first principles, starting from Maxwell's equations and the Helmholtz equation, and implemented as a reusable Python function. Its correctness was confirmed through three independent checks: agreement with the analytic Airy diffraction pattern for a circular aperture, agreement with the analytic Fraunhofer pattern for a single slit, and a reversibility test confirming that forward-and-backward propagation returns the original field to within numerical precision ($\sim 10^{-16}$). Additional tests with a double-slit aperture and an off-axis aperture further confirmed correct behavior and surfaced known limitations of the method related to finite-grid sampling. Together, these results establish a validated, trustworthy propagation engine suitable as the foundation for further work.

The validated propagation engine can be used as the forward model of a trainable system. The angular spectrum method was reimplemented in PyTorch in double precision, and it reproduced the Phase 1 Airy and single-slit results. A single phase mask, initialized to zero, was then trained by gradient descent to route light from two input positions to two output regions. The loss fell from 1.0 (chance-level routing) to 0.0053, and over 99.8% of each input's light ended up in its target region, compared with 50% before training. This is a minimal demonstration that a diffractive layer can be trained like a neural network layer, using only free-space propagation and phase modulation. The mask was trained and tested on the same two inputs, so the result shows that the training pipeline works, not that the mask generalizes to new inputs.